

Exercise 1 — Recursive Sum of an Array

Write a recursive function to calculate the sum of the elements in an integer array.

Example: For [3, 5, 7, 9], the function should return 24.

Exercise 2 — Recursive Fibonacci

Write a recursive function to calculate the nth Fibonacci number. Call the function to compute and print the 8th Fibonacci number.

Exercise 3 — Recursive Tribonacci

Write a recursive function to calculate the nth Tribonacci number, with the convention: - $T_0 = 0$, $T_1 = 1$, $T_2 = 1$ - $T_n = T(n-1) + T(n-2) + T(n-3)$, for $n \geq 3$

Call the function to compute and print $T(9)$.

Optional optimization: Add memoization to improve performance.

Exercise 4 — Recursive GCD with Memoization

Use recursion to find the greatest common divisor (GCD) of two numbers, storing intermediate results using a HashMap to avoid recomputation (memoization).

- Input: Two integers.
- Output: The GCD of the two numbers.

Exercise 5 — Fibonacci with Memoization

Write a program to calculate the nth Fibonacci number using **recursion**, storing previously computed values in a HashMap (memoization) to avoid recalculating them.

- Input: An integer n.
- Output: The nth Fibonacci number.

Exercise 6 — Reversing a Queue Using Recursion

Write a program that uses **recursion** to reverse the elements in a **queue**, without using any loop construct.